

CSE 220

Data Structures

LAB 01: Arrays and Linked List

Anwarul Bashir Shuaib [AWBS]
Lecturer
Department of Computer Science and Engineering
BRAC University

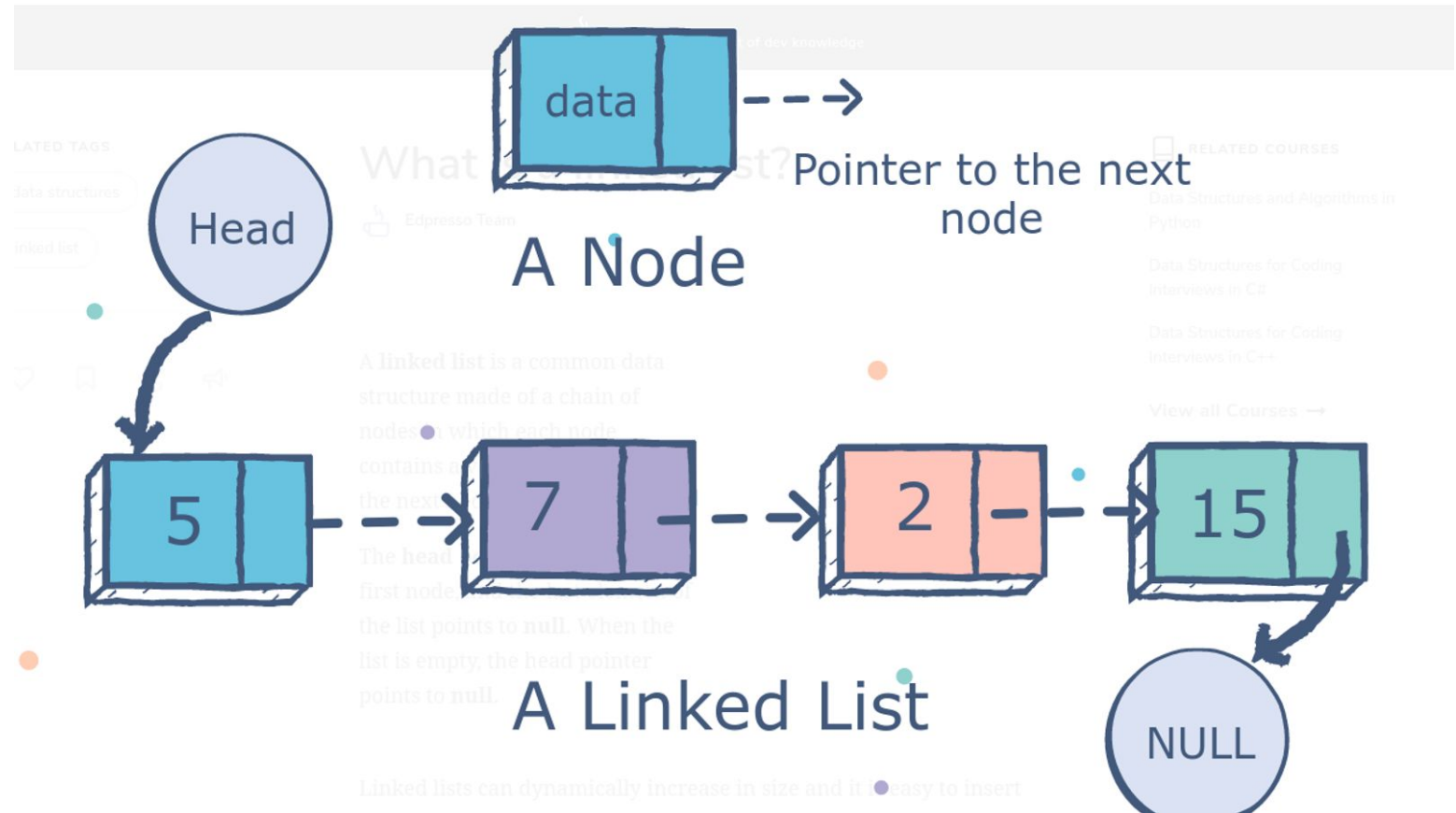


Linked List Overview

A node is the building block for linked lists.

Contains two key things:

1. **Data**
2. **Reference to next node**

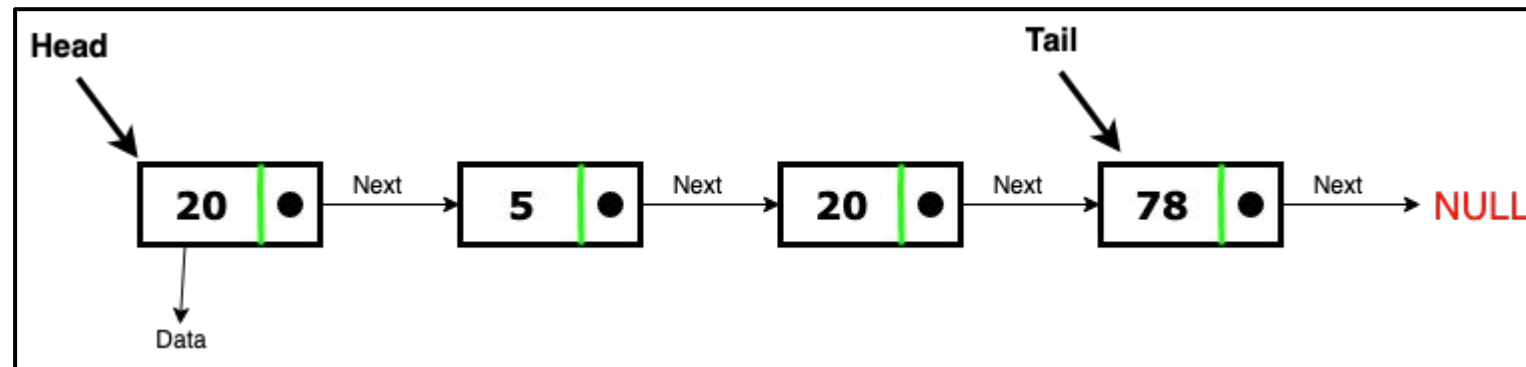


<https://dev.to/tastaslim/an-introduction-to-linked-list-1bmp>

Linked List Overview

The first element is often called “**head**”

The last element is often called “**tail**”



Node

Linked List

```
static class Node{  
    int elem;  
    Node next;  
  
    public Node(int elem){  
        this.elem = elem;  
        this.next = null;  
    }  
}
```

```
private Node head;
```

Linked List Creation

- Create a linked list from these elements: 21, 43, 31, 10

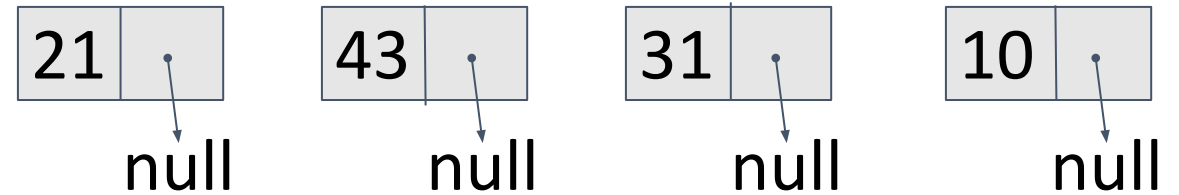
```
Node n1 = new Node( elem: 21);  
Node n2 = new Node( elem: 43);  
Node n3 = new Node( elem: 31);  
Node n4 = new Node( elem: 10);
```



Linked List Creation

- Create a linked list from these elements: 21, 43, 31, 10

```
Node n1 = new Node( elem: 21);  
Node n2 = new Node( elem: 43);  
Node n3 = new Node( elem: 31);  
Node n4 = new Node( elem: 10);
```

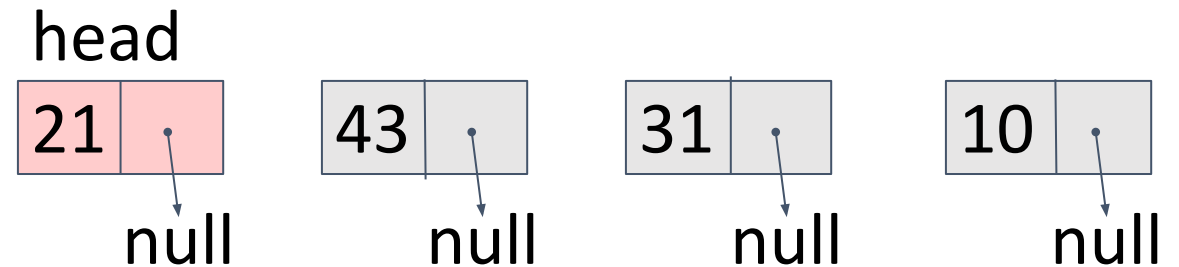


Linked List Creation

- Create a linked list from these elements: 21, 43, 31, 10

```
Node n1 = new Node( elem: 21);  
Node n2 = new Node( elem: 43);  
Node n3 = new Node( elem: 31);  
Node n4 = new Node( elem: 10);
```

```
Node head = n1;
```

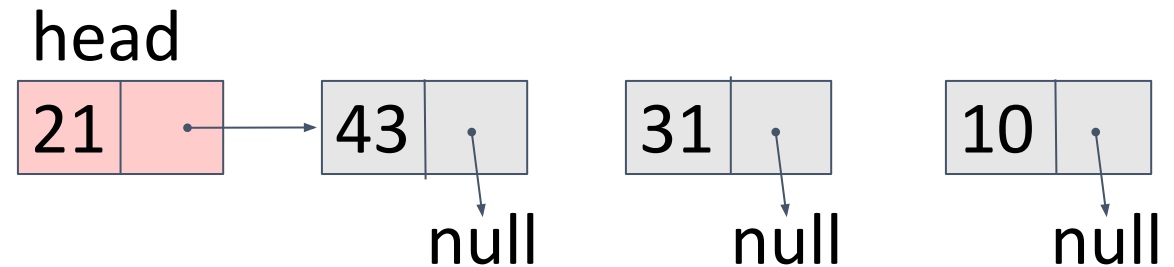


Linked List Creation

- Create a linked list from these elements: 21, 43, 31, 10

```
Node n1 = new Node( elem: 21);
Node n2 = new Node( elem: 43);
Node n3 = new Node( elem: 31);
Node n4 = new Node( elem: 10);
```

```
Node head = n1;
n1.next = n2;
```

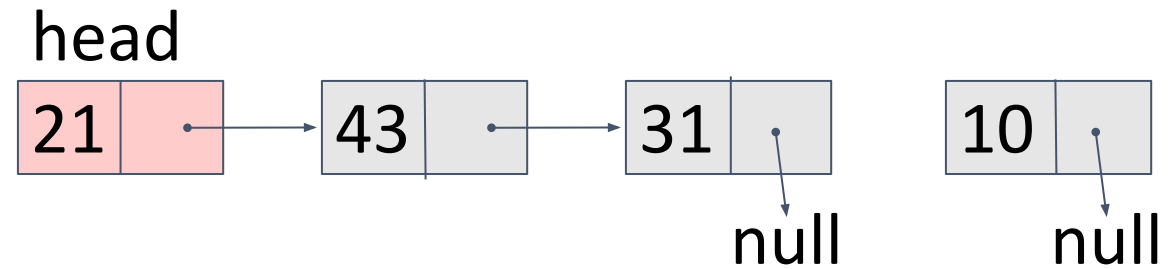


Linked List Creation

- Create a linked list from these elements: 21, 43, 31, 10

```
Node n1 = new Node( elem: 21);
Node n2 = new Node( elem: 43);
Node n3 = new Node( elem: 31);
Node n4 = new Node( elem: 10);
```

```
Node head = n1;
n1.next = n2;
n2.next = n3;
```

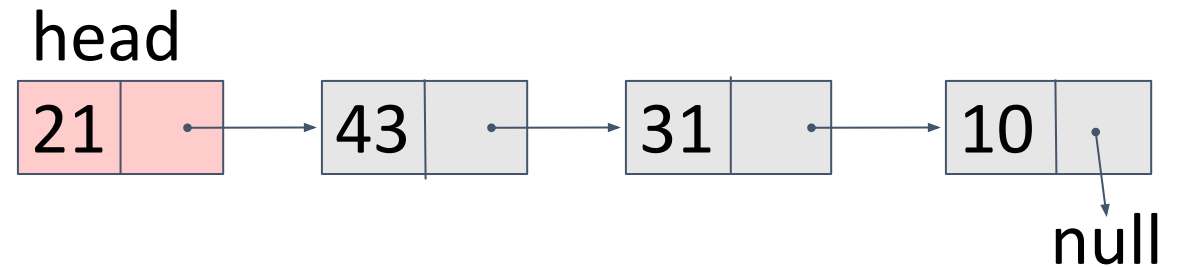


Linked List Creation

- Create a linked list from these elements: 21, 43, 31, 10

```
Node n1 = new Node( elem: 21);  
Node n2 = new Node( elem: 43);  
Node n3 = new Node( elem: 31);  
Node n4 = new Node( elem: 10);
```

```
Node head = n1;  
n1.next = n2;  
n2.next = n3;  
n3.next = n4;
```

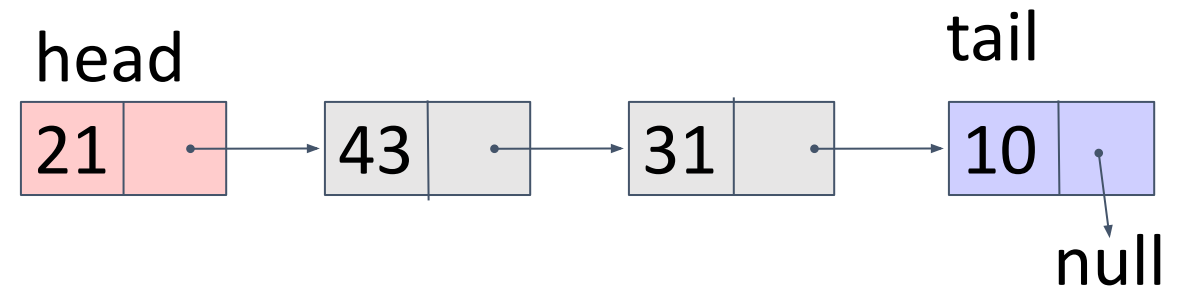


Linked List Creation

- Create a linked list from these elements: 21, 43, 31, 10

```
Node n1 = new Node( elem: 21);
Node n2 = new Node( elem: 43);
Node n3 = new Node( elem: 31);
Node n4 = new Node( elem: 10);
```

```
Node head = n1;
n1.next = n2;
n2.next = n3;
n3.next = n4;
Node tail = n4;
```



Linked List Creation

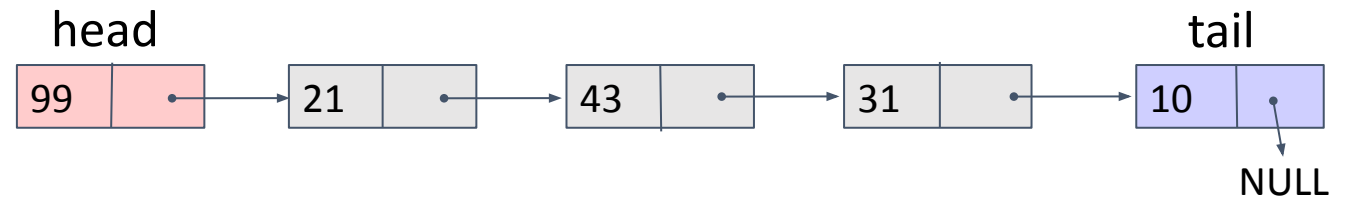
- Create a linked list from an array

arr = [21, 43, 31, 10]

```
// 1. Create a Linked List from an array
public void createFromArray(int[] arr) {
    if (arr == null || arr.length == 0) return;
    head = new Node(arr[0]);
    Node current = head;
    for (int i = 1; i < arr.length; i++) {
        current.next = new Node(arr[i]);
        current = current.next;
    }
}
```

$O(n)$

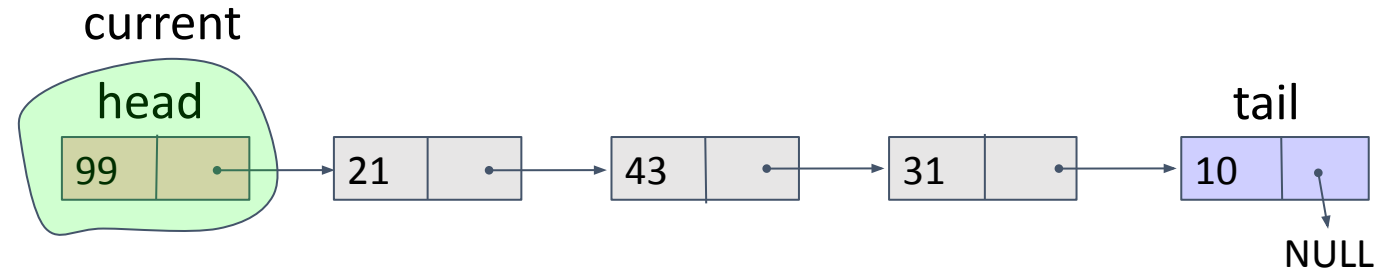
Sequential Traversal



```
// 2. Iteration of the linked list
public void iterate() {
}
}
```

Output:

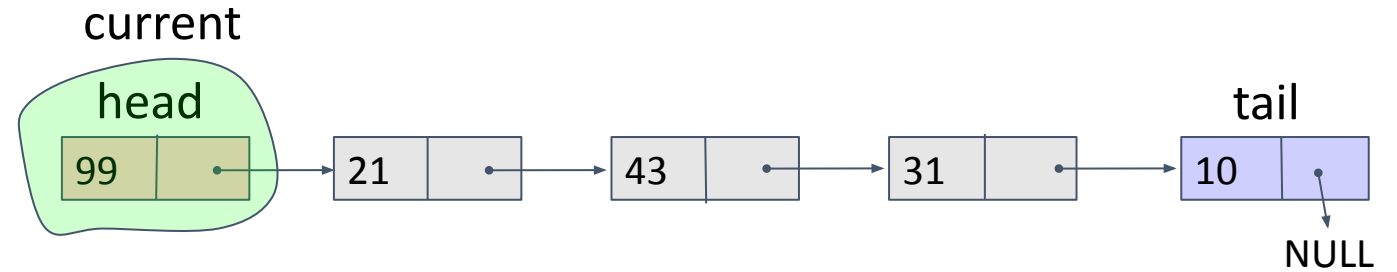
Sequential Traversal



```
// 2. Iteration of the linked list
public void iterate() {
    Node current = head;
}
}
```

Output:

Sequential Traversal

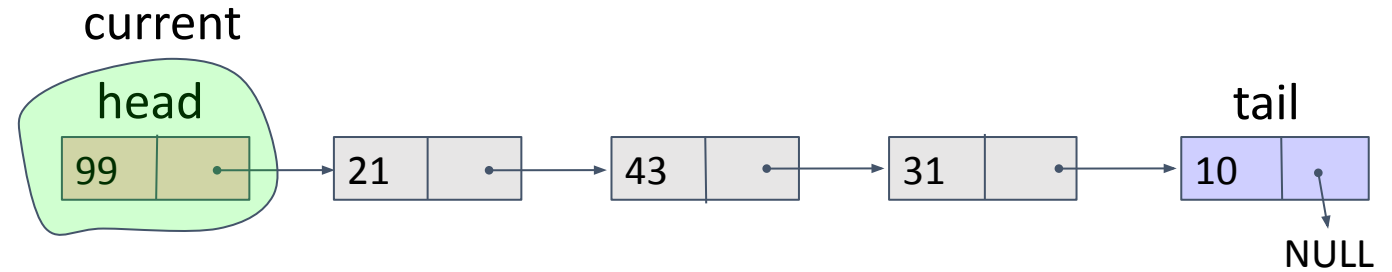


```
// 2. Iteration of the linked list
public void iterate() {
    Node current = head;
    while (current != null) {

    }
}
```

Output:

Sequential Traversal

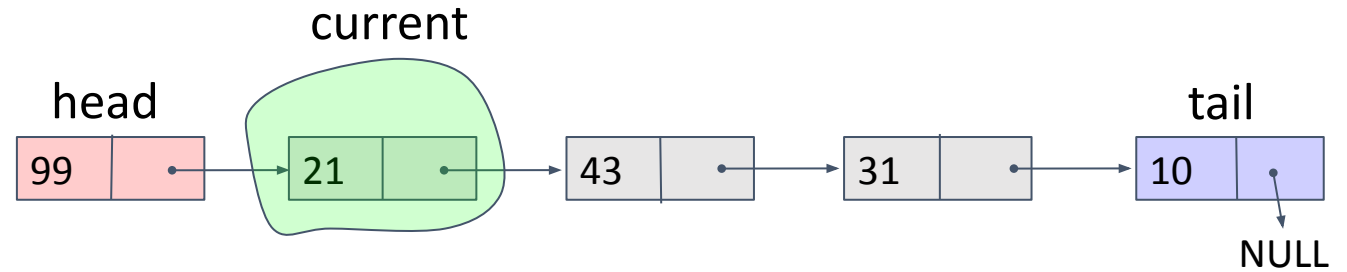


```
// 2. Iteration of the linked list
public void iterate() {
    Node current = head;
    while (current != null) {
        System.out.print(current.elem + " -> ");
    }
}
```

Output:

99->

Sequential Traversal

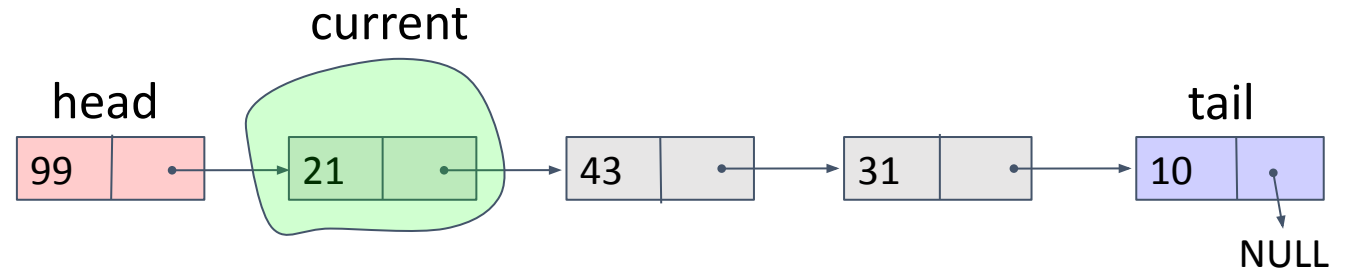


```
// 2. Iteration of the linked list
public void iterate() {
    Node current = head;
    while (current != null) {
        System.out.print(current.elem + " -> ");
        current = current.next;
    }
}
```

Output:

99->

Sequential Traversal

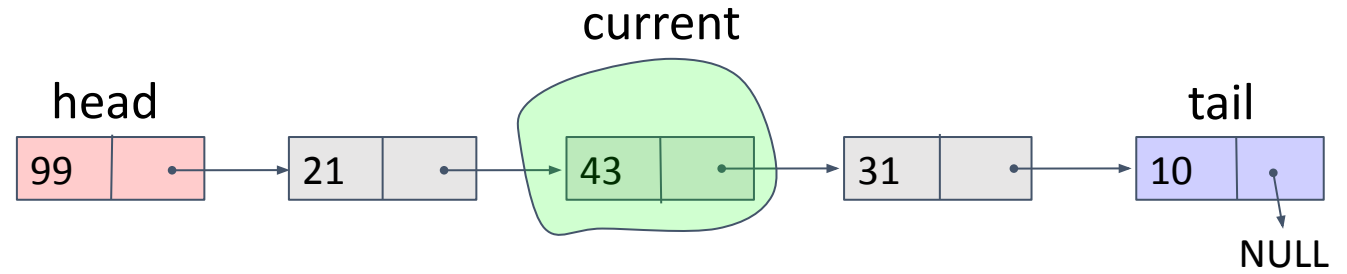


```
// 2. Iteration of the linked list
public void iterate() {
    Node current = head;
    while (current != null) {
        System.out.print(current.elem + " -> ");
        current = current.next;
    }
}
```

Output:

99->21->

Sequential Traversal

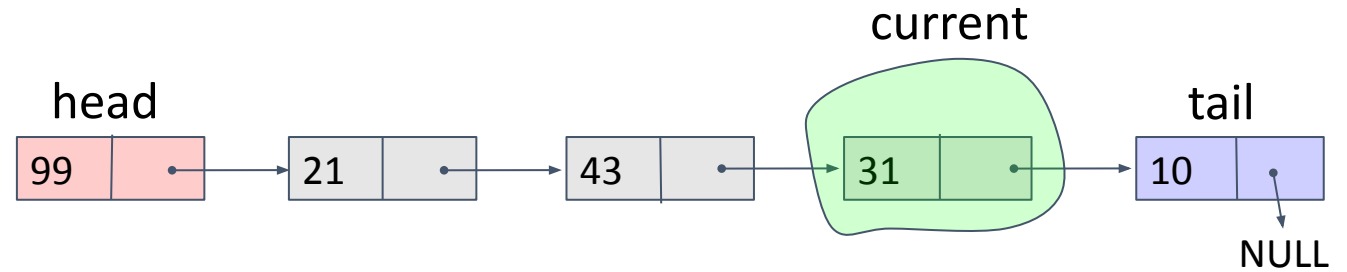


```
// 2. Iteration of the linked list
public void iterate() {
    Node current = head;
    while (current != null) {
        System.out.print(current.elem + " -> ");
        current = current.next;
    }
}
```

Output:

99->21->43

Sequential Traversal

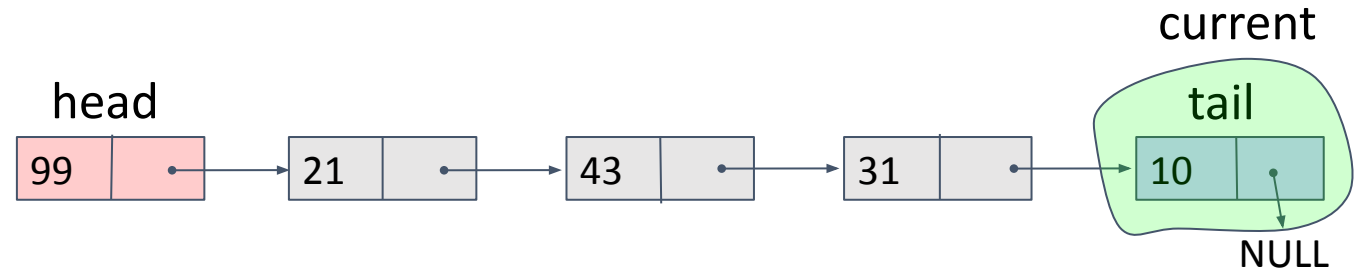


```
// 2. Iteration of the linked list
public void iterate() {
    Node current = head;
    while (current != null) {
        System.out.print(current.elem + " -> ");
        current = current.next;
    }
}
```

Output:

99->21->43->31

Sequential Traversal

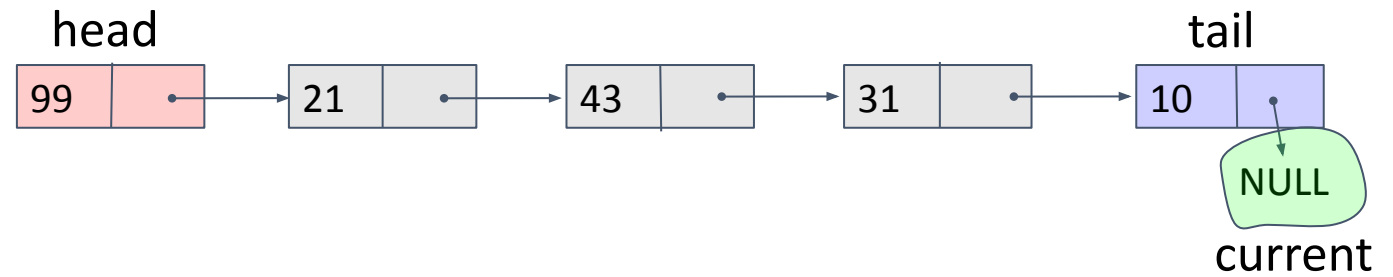


```
// 2. Iteration of the linked list
public void iterate() {
    Node current = head;
    while (current != null) {
        System.out.print(current.elem + " -> ");
        current = current.next;
    }
}
```

Output:

99->21->43->31->10->

Sequential Traversal



```
// 2. Iteration of the linked list
public void iterate() {
    Node current = head;
    while (current != null) {
        System.out.print(current.elem + " -> ");
        current = current.next;
    }
}
```

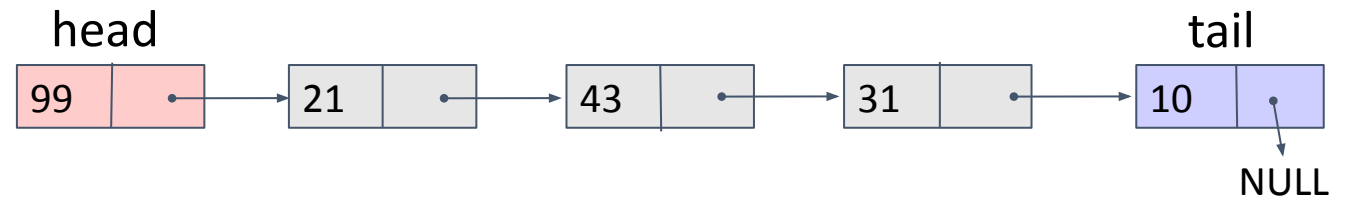
Termination condition

Output:

99->21->43->31->10->

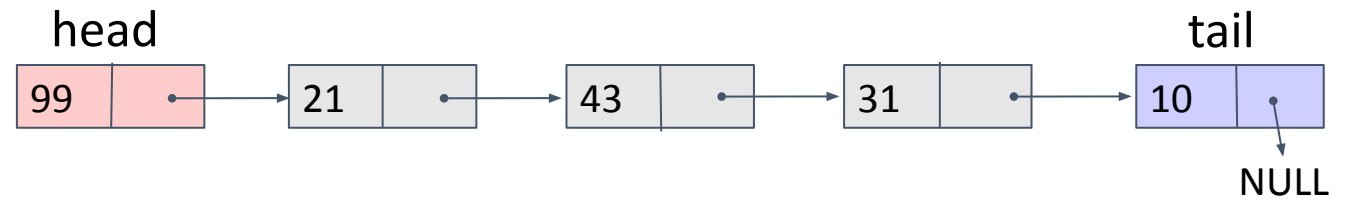


Node Count



```
// 3. Count the items in the linked list
public int count() {
}
}
```

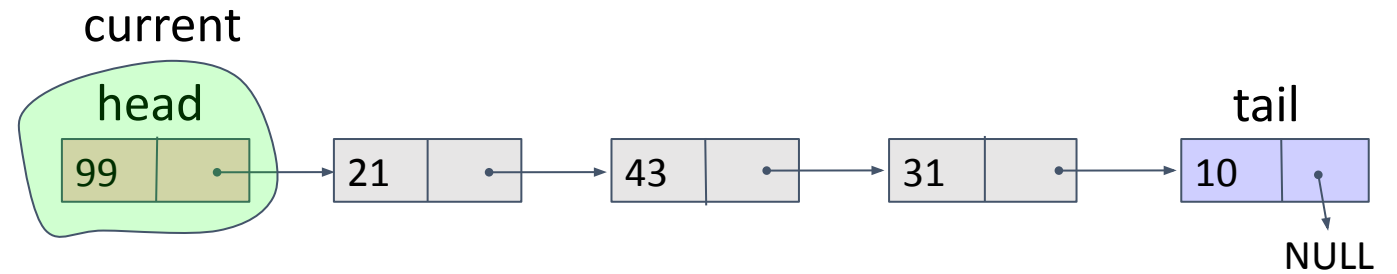
Node Count



```
// 3. Count the items in the linked list
public int count() {
    int count = 0;
}
}
```

count = 0

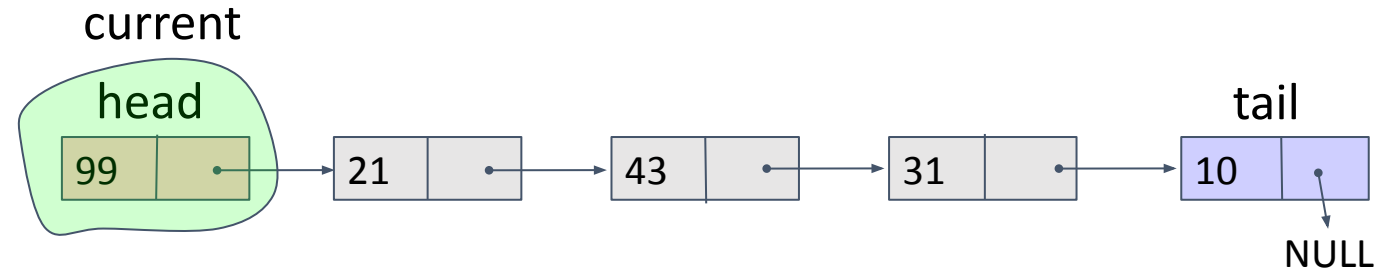
Node Count



```
// 3. Count the items in the linked list
public int count() {
    int count = 0;
    Node current = head;
}
```

count = 0

Node Count

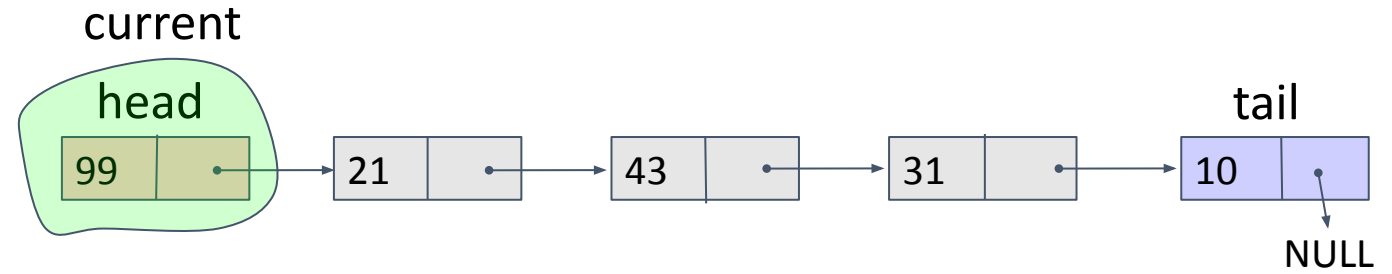


```
// 3. Count the items in the linked list
public int count() {
    int count = 0;
    Node current = head;
    while (current != null) {

    }
}
```

count = 0

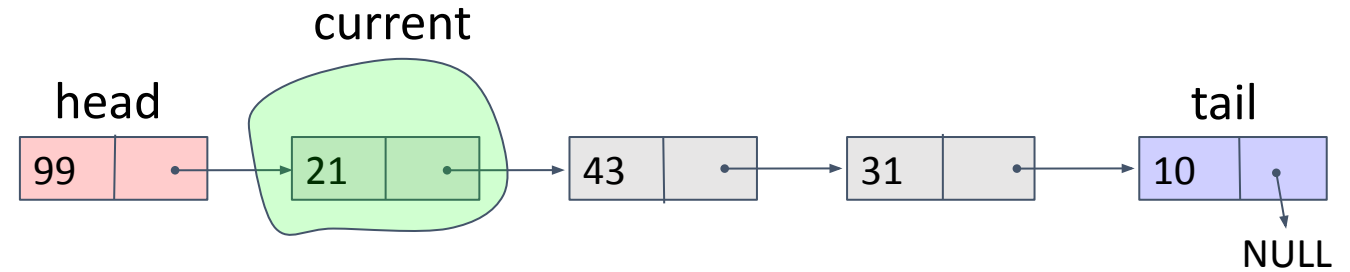
Node Count



```
// 3. Count the items in the linked list
public int count() {
    int count = 0;
    Node current = head;
    while (current != null) {
        count++;
    }
}
```

count = 1

Node Count

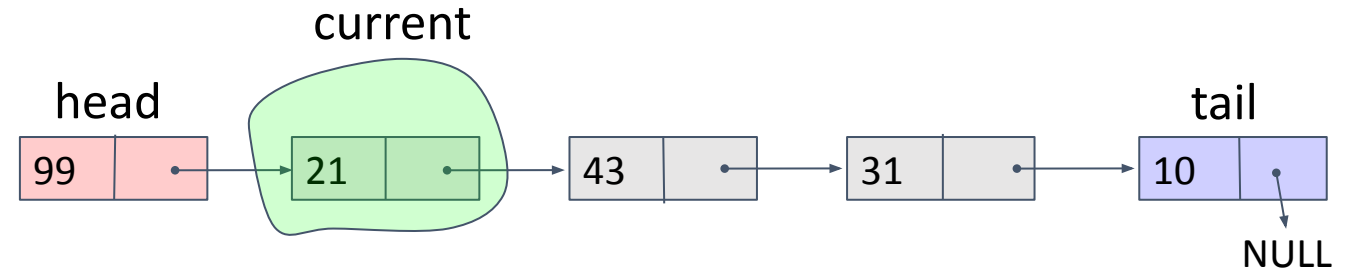


```
// 3. Count the items in the linked list
public int count() {
    int count = 0;
    Node current = head;
    while (current != null) {
        count++;
        current = current.next;
    }
}
```

count = 1



Node Count

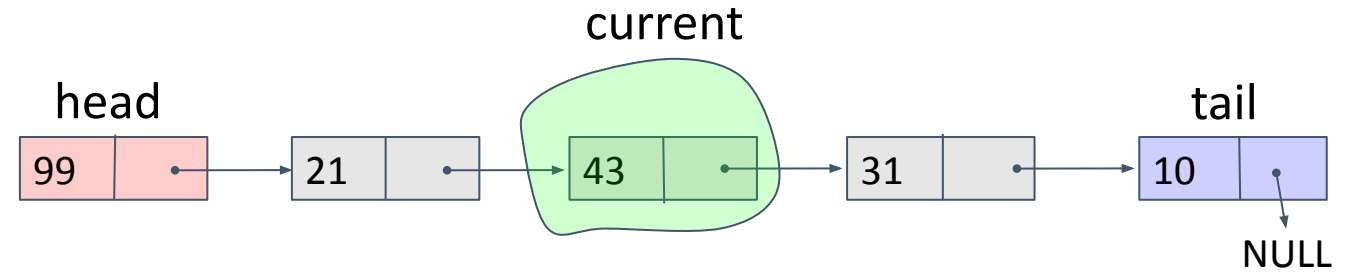


```
// 3. Count the items in the linked list
public int count() {
    int count = 0;
    Node current = head;
    while (current != null) {
        count++;
        current = current.next;
    }
}
```

count = 2



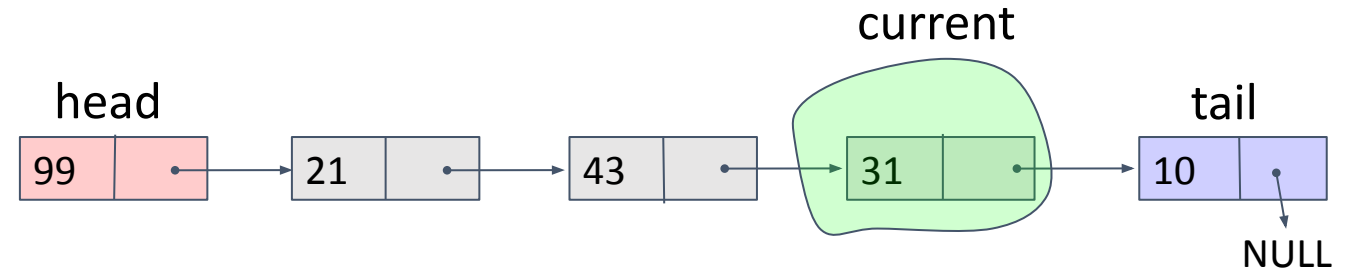
Node Count



count = 3

```
// 3. Count the items in the linked list
public int count() {
    int count = 0;
    Node current = head;
    while (current != null) {
        count++;
        current = current.next;
    }
}
```

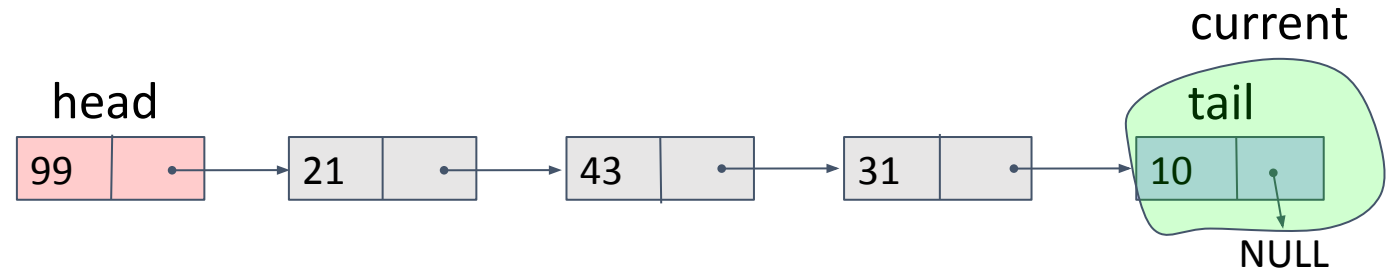
Node Count



```
// 3. Count the items in the linked list
public int count() {
    int count = 0;
    Node current = head;
    while (current != null) {
        count++;
        current = current.next;
    }
}
```

count = 4

Node Count

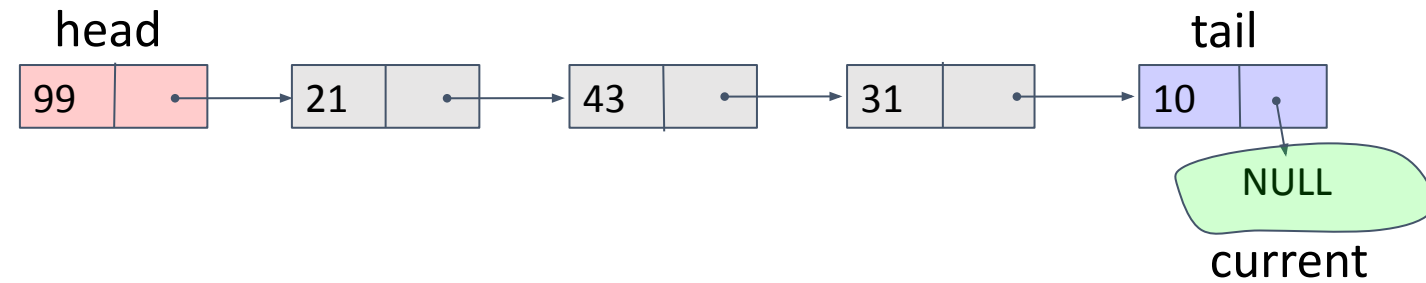


```
// 3. Count the items in the linked list
public int count() {
    int count = 0;
    Node current = head;
    while (current != null) {
        count++;
        current = current.next;
    }
}
```

count = 5



Node Count



```
// 3. Count the items in the linked list
public int count() {
    int count = 0;
    Node current = head;
    while (current != null) {
        count++;
        current = current.next;
    }
    return count;
}
```

count = 5

Termination



Element Access

```
// 4. Retrieve index of an element
public int indexOf(int elem) {
    int index = 0;
    Node current = head;
    while (current != null) {
        if (current.elem == elem) {
            return index;
        }
        current = current.next;
        index++;
    }
    return -1; // Element not found
}
```

$O(n)$

```
// 5. Retrieve a node from an index
public Node getNode(int index) {
    int currentIndex = 0;
    Node current = head;
    while (current != null) {
        if (currentIndex == index) {
            return current;
        }
        current = current.next;
        currentIndex++;
    }
    return null; // Index out of bounds
}
```

$O(n)$